

| 第十一周作业 -1

| 第十一周作业 -1

| 基础作业

| 1.1 YOLO 推理环境与课堂 Demo 复现

按照操作手册，完成 YOLO 推理环境搭建，并复现课堂 Demo。

1. 进入课程资料目录并激活环境：

```
cd ~/robot_yolo_democonda activate robot_yolo
```

2. 检查 Ultralytics 是否能正常运行：

```
yolo checks
```

3. 使用 YOLO11n 对课堂主图进行单图推理：

```
yolo predict model=models/yolo11n.pt source=images/soccer_practice_field.png conf=0.25 project=outputs name=cli_predict exist_ok=True
```

4. 运行 Python 脚本读取检测结果：

```
python scripts/detect_image.py
```

提供以下验证材料：

- `yolo checks` 运行成功截图 1 张；
- CLI 推理终端输出截图 1 张，能看到 `1 person, 1 sports ball` 或类似检测结果；
- YOLO 生成的检测结果图 1 张；
- Python 脚本输出截图 1 张，能看到 `class_name`、`confidence`、`bbox_xyxy`。

1. Yolo 运行截图：

```
(robot_yolo_2) [sunzj@SH2 ~/Robot/OpenCV/Lec11/robot_yolo_demo]$ yolo check
Ultralytics 8.4.33 🚀 Python-3.10.20 torch-2.10.0 CPU (AMD EPYC 7K83 64-Core Processor)
Setup complete ✅ (2 CPUs, 7.1 GB RAM, 64.1/147.5 GB disk)

OS                Linux-5.4.0-196-generic-x86_64-with-glibc2.31
Environment       Linux
Python            3.10.20
Install           pip
Path              /home/sunzj/miniconda3/envs/robot_yolo_2/lib/python3.10/site-packages/ultralytics
RAM               7.14 GB
Disk              64.1/147.5 GB
CPU               AMD EPYC 7K83 64-Core Processor
CPU count         2
GPU               None
GPU count         None
CUDA              None

numpy              ✅ 1.26.4>=1.23.0
matplotlib         ✅ 3.10.9>=3.3.0
opencv-python     ✅ 4.13.0>=4.6.0
pillow            ✅ 12.2.0>=7.1.2
pyyaml            ✅ 6.0.3>=5.3.1
requests          ✅ 2.33.1>=2.23.0
scipy             ✅ 1.15.3>=1.4.1
torch             ✅ 2.10.0>=1.8.0
torch             ✅ 2.10.0!=2.4.0,>=1.8.0; sys_platform == "win32"
torchvision       ✅ 0.25.0>=0.9.0
psutil            ✅ 7.0.0>=5.8.0
polars            ✅ 1.39.0>=0.20.0
1. ultralytics-thop ✅ 2.0.18>=2.0.18
```

2. CLI 运行截图：

```
(robot_yolo_2) [sunzj@SH2 ~/Robot/OpenCV/Lec11/robot_yolo_demo]$ yolo predict model=models/yolo11n.pt source=images/soccer_practice_field.png conf=0.25 project=outputs name=cli_predict exist_ok=True
Ultralytics 8.4.33 🚀 Python-3.10.20 torch-2.10.0 CPU (AMD EPYC 7K83 64-Core Processor)
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs

image 1/1 /home/sunzj/Robot/OpenCV/Lec11/robot_yolo_demo/images/soccer_practice_field.png: 448x640 1 person, 1 sports ball, 129.3ms
Speed: 3.5ms preprocess, 129.3ms inference, 1.0ms postprocess per image at shape (1, 3, 448, 640)
Results saved to /home/sunzj/Robot/OpenCV/Lec11/robot_yolo_demo/runs/detect/outputs/cli_predict
1. 🌟 Learn more at https://docs.ultralytics.com/modes/predict
```

3. YOLO 生成检测结果图：



```
1.
4. Python 脚本输出截图：
(robot_yolo_2) [sunzj@SH2 ~/Robot/OpenCV/Lec11/robot_yolo_demo]$ python scripts/detect_image.py

image 1/1 /home/sunzj/Robot/OpenCV/Lec11/robot_yolo_demo/images/soccer_practice_field.png: 448x640 1 person, 1 s
ports ball, 144.5ms
Speed: 2.4ms preprocess, 144.5ms inference, 1.0ms postprocess per image at shape (1, 3, 448, 640)
Results saved to /home/sunzj/Robot/OpenCV/Lec11/robot_yolo_demo/outputs/python_predict
image: images/soccer_practice_field.png
detections: 2
-----
[0] class_name=sports ball
    confidence=0.959
    bbox_xyxy=(602.7, 416.7, 976.5, 786.1)
[1] class_name=person
    confidence=0.906
    bbox_xyxy=(150.7, 0.8, 615.4, 641.6)
-----
1. visual result saved to: outputs/python_predict
```

1.2 足球识别稳定性分析

使用同一个预训练模型和同一个阈值，测试资料包中的多张图片：

```
yolo predict model=models/yolo11n.pt source=images conf=0.25 project=outputs name=homework_soccer_check exist_ok=True
```

如果某些图片没有明显检测结果，可以把置信度阈值临时降低到 0.10 再观察：

```
yolo predict model=models/yolo11n.pt source=images conf=0.10 project=outputs name=homework_soccer_check_conf010 exist_ok=True
```

测试后回答以下问题：

- 1. 哪些图片被识别成了 sports ball？对应置信度大约是多少？
- 2. 哪些图片没有识别出足球？哪些图片被识别成了其他类别？
- 3. 降低 conf 后，检测结果有什么变化？是更容易出现目标框，还是误检也变多？
- 4. 比较 soccer_practice_field.png 和 color_ball_clean.png，它们在背景、纹理、上下文上有什么不同？
- 5. 为什么同样是“球”，预训练 YOLO 对不同图片的识别稳定性会不同？

1. 6 张图片的识别情况如下：

图片名称	识别后的对象	置信度
color_ball_clean.png	1 orange	0.73
color_ball_floor_bg.png	1 orange	0.49
soccer_grass.png	1 sports ball, 1 mouse	0.44, /
soccer_indoor_floor.png	1 banana	0.26
soccer_practice_field.png	1 sports ball, 1 person	0.96, 0.91
soccer_studio_closeup.png	no detections	/

2. 如上所述，`soccer_grass.png` 和 `soccer_practice_field.png` 识别出了足球，而 `color_ball_clean.png`，`color_ball_floor_bg.png`，`soccer_indoor_floor.png` 识别成了其他物体，`soccer_studio_closeup.png` 没有识别出来。
3. 降低 conf 到 0.1 之后，检测结果如下。检测结果没有明显变化，第四幅图中增加了 dining table 的识别结果。

图片名称	识别后的对象	置信度
color_ball_clean.png	1 orange	0.73
color_ball_floor_bg.png	1 orange	0.49
soccer_grass.png	1 sports ball, 1 mouse	0.44
soccer_indoor_floor.png	1 banana, dining table	0.26, 0.14
soccer_practice_field.png	1 sports ball, 1 person	0.96, 0.91
soccer_studio_closeup.png	no detections	/

1. 这两张图片在背景、纹理和上下文（语境）上存在非常显著的差异:
- 背景方面：`color_ball_clean.png` 采用单调且平滑的灰白色渐变背景，类似虚拟渲染空间或影棚无缝墙；背景中没有任何噪点、阴影（除球体下方极轻微的接触阴影外）或其它物理实体，完全凸显了中心的球体。`soccer_practice_field.png` 背景为户外的绿茵足球场，包含近处清晰的草坪、中景虚化的足球门框，以及远处的树木和围栏。受到真实日光照射，球体和球员的腿部在草地上投下了明确的投影，且背景具有由于镜头光圈产生的景深虚化效果。

• 纹理方面：`color_ball_clean.png` 球体表面呈完美的数学圆弧，纹理为平滑的纯橙色渐变，没有任何粗糙度、接缝、反光颗粒或物理划痕。整个图像的熵较低，纹理细节极少。`soccer_practice_field.png` 为经典的黑白拼接结构，表面有清晰的皮革缝线、合成材质的微小颗粒感，以及由于光照产生的非均匀反射。地面由无数根草丝组成的草坪纹理，细节饱满，明暗交织。包含球员球鞋的皮革纹理、鞋带细节、袜子的针织纵向条纹，以及腿部皮肤的纹理。

• 上下文/语境 (Context) 方面：`color_ball_clean.png`图片没有讲述任何“故事”或现实活动。它仅是一个几何球体。`soccer_practice_field.png` 图像传达了明确的上下文——“足球运动/控球训练”。球、球员（腿部、球鞋）、草坪和远处的球门构成了一个逻辑严密的运动场景。
2. 预训练 YOLO（通常在 COCO 数据集上训练）对这两张图片中的“球”表现出不同的识别稳定性（甚至可能出现置信度极低或漏检/误检），主要是由于深度学习目标检测算法的决策机制所致。这主要由以下几个核心原因造成：

1. 训练集数据分布与领域鸿沟 (Domain Gap)

- **COCO 数据集中的定义**：预训练 YOLO 的 `sports ball`（运动球类）类别主要由足球、篮球、网球、棒球等构成。
- **足球的契合度**：`soccer_practice_field.png` 中的球是典型的黑白拼接足球，这是 `sports ball` 类别的经典代表。模型在训练中见过成千上万个类似的足球，因此其特征提取器能完美契合。
- **橙色球的偏离度**：`color_ball_clean.png` 中的球是一个纯橙色、无纹理的几何球体。在 COCO 训练集中，极少有这种“无纹理、纯单色”的球体作为运动球出现。它更偏向于玩具、3D 渲染图或台球，导致了**领域偏移 (Domain Shift)**，模型无法对其产生很强的分类激活。

2. 上下文语义信息的缺失 (Lack of Contextual Clues)

现代目标检测网络（如 YOLO 中的路径聚合网络 PANet 或特征金字塔 FPN）在进行预测时，不仅看物体本身，还会结合物体周围的环境（上下文）进行联合推理。

- **足球的强上下文**：在 `soccer_practice_field.png` 中，绿色的草坪、穿着球鞋和长袜的球员腿部，甚至远处的球门，都为模型提供了强烈的“足球场”上下文约束。模型“知道”在人脚下的草坪上，这个圆形的黑白物体极大概率是足球。
- **橙色球的零上下文**：`color_ball_clean.png` 只有一具悬空的球体和纯白背景。没有地面、没有人、没有运动器材。缺乏上下文线索迫使模型只能进行“孤立”的特征推理，这会显著降低模型输出的置信度（Confidence）。

3. 局部纹理与高频特征的强弱

卷积神经网络（CNN）对边缘、局部纹理和对比度变化非常敏感。

- **足球的强特征**：足球黑白相间的斑块构成了极强的高频对比度纹理。这种“五边形/六边形黑白网格”在卷积操作中会产生非常清晰且独特的激活图（Activation Map）。
- **橙色球的弱特征**：橙色球表面非常平滑，除了边缘的圆形弧度外，只有平缓的渐变（低频信号）。这种平滑渐变很容易在网络深层传递中被“平滑掉”或丢失，使得网络很难提取出能够进行特异性分类的局部特征。

4. 类别多义性与混淆 (Class Ambiguity)

因为特征的单一性，橙色渐变球在模型眼中具有极高的多义性：

- 它很容易与 COCO 数据集中的其他圆形/球状物体混淆，例如：

• 橙子 (orange) 或 苹果 (apple)（特别是它的橙黄色调和圆形轮廓）

• 气球 (balloon)

• 飞盘 (frisbee)（如果视角变化）
- 而黑白相间的足球几乎不可能被混淆为食物或其他日常用品。这种多义性会导致 YOLO 在预测橙色球时，不同类别之间的 Softmax 得分非常接近，从而拉低了整体的置信度。

5. 尺度匹配问题 (Scale Mismatch)

- YOLO 在设计时会使用先验框 (Anchor Boxes) 或特定尺度的感受野来匹配不同大小的目标。
- `color_ball_clean.png` 中，球体几乎占据了画面的中央，比例巨大且周围没有任何物理参照。对于通常在画面中占比为中小尺寸的 `sports ball` 来说，这种超常规尺度可能会偏离 YOLO 预设的尺度检测范围，导致检测框定位不稳定。

1.3 YOLO 输出字段理解

结合课堂 Demo 和 `scripts/detect_image.py`，回答以下问题：

1. `class_name` 表示什么？为什么足球在本次模型中通常显示为 `sports ball`，而不是 `football`？
2. `confidence` 表示什么？置信度高是否一定代表检测结果完全正确？
3. `bbox_xyxy = (x1, y1, x2, y2)` 中四个数分别代表什么？
4. CLI 输出中的 `preprocess`、`inference`、`postprocess` 分别表示什么阶段？

5. NMS 的作用是什么？为什么一个目标可能会先产生多个候选框，最后只保留一个框？

- `class_name` 表示模型检测到的目标物体的类别名称（字符串形式，如 `"person"`、`"sports ball"` 等）。为什么足球在本次模型中通常显示为 `sports ball`，而不是 `football`：
 - 所使用的预训练模型和数据集：代码中加载的 `yolo11n.pt`（第 8 行）是基于 **COCO** 数据集 进行预训练的官方默认模型。
 - COCO** 类别设计的局限性：COCO 数据集共定义了 80 个物体类别。为了简化分类并提高通用性，COCO 没有将各种球类进行细分（即数据集中没有 `football`、`basketball`、`tennis ball` 等独立类别），而是将足球、篮球、网球、棒球等所有用于体育运动的球状物统一合并为了一个大类，命名为 `sports ball`（类别 ID 通常为 32）。
 - 无法输出未学习的类别：目标检测模型只能识别它在训练阶段学习过的类别。由于预训练的 YOLO 从未在标注有 `football` 的数据集上进行训练，因此即使它在图像中看到了一个足球，它也只能将其归类为它所知道的、最贴近的类别——`sports ball`。
- `confidence`（置信度分数）是一个介于 `0.0` 到 `1.0` 之间的概率值。它表示模型对自己预测出的“该边界框（Bounding Box）中包含某个特定类别物体”的把握程度（信心大小）。在 YOLO 的计算逻辑中，置信度通常由两部分相乘得到： $\text{Confidence} = P(\text{Class}) \times \text{Objectness}$

- $P(\text{Class})$ （类别概率）：模型认为框内物体属于某个特定类别（如 `sports ball`）的概率。
- Objectness（含物概率/定位质量）：模型认为这个框内确实存在“一个物体”（而不是背景）的概率，同时结合了预测框与真实物体的重合程度（IoU，交并比）。
- 简单来说，`confidence = 0.95` 意味着模型有 95% 的把握确定这里有一个它识别出的物体。

置信度高是否一定代表检测结果完全正确？不一定。高置信度并不等同于绝对正确。在实际应用中，模型可能会以非常高的置信度（例如 `0.90` 以上）做出错误的判断。主要原因有以下几点：

① 模型“盲目自信”（Overconfidence）与数据偏差

深度神经网络（包括 YOLO）容易对与训练数据局部特征相似的未见过物体产生“盲目自信”。

- 示例：如果图像中出现了一个圆形的黄橙色气球，由于其圆形轮廓和鲜艳颜色类似于网球或篮球，YOLO 可能会以 `0.92` 的高置信度将其误检测为 `sports ball`。模型并不知道“气球”这一概念（如果 COCO 中没有），只能用自己最自信的已知类别去强行解释。

② 背景或纹理的巧合混淆

图像中的光影、纹理等噪声可能恰好组合成了某种物体的特征，触发了神经元的强烈激活。

- 示例：树叶间的光影斑驳可能正好组成了类似于猫咪花纹的图案，模型可能会以很高的置信度在这个区域框出 `cat`（俗称“幻觉”或“虚警”）。

③ 上下文偏见（Contextual Bias）

模型过度依赖物体周围的背景进行判断，导致在非典型场景下出错，或在典型场景下过度联想。

- 示例：如果在足球场草地上放一个圆形的西瓜，因为周围满是草地和球员（极强的足球上下文），模型可能会以极高的置信度将西瓜误判为 `sports ball`。

④ 定位不准但分类极度自信

有时模型成功认出了物体（如人），但画出的边界框只包含了小半个身体或者把两个人框在了一起，此时虽然 `confidence` 很高，但从“目标检测定位”的角度来看，该检测结果依然是不完全正确的。

- `bbox_xyxy = (x1, y1, x2, y2)` 代表的是边界框（Bounding Box）的对角像素坐标格式。这四个数字的具体含义如下：
 - `x1`：边界框左上角顶点的 横坐标（X 轴坐标）。
 - `y1`：边界框左上角顶点的 纵坐标（Y 轴坐标）。
 - `x2`：边界框右下角顶点的 横坐标（X 轴坐标）。
 - `y2`：边界框右下角顶点的 纵坐标（Y 轴坐标）。
- 在 YOLO（以及绝大多数深度学习目标检测）的运行输出中，`preprocess`、`inference` 和 `postprocess` 代表了图像进入系统到最终输出检测结果的三个核心处理阶段。下面是每个阶段的具体职责和所做的工作：
 - Preprocess (预处理阶段)**：将输入的原始图像（如各种尺寸、格式、色彩空间的图片）转换为模型能够接收的标准张量（Tensor）输入格式。核心操作包括：
 - 尺寸调整 (**Resizing / Letterboxing**)：YOLO 模型需要固定大小的输入（如 640×640 像素）。预处理会将原始图片缩放到目标尺寸，通常会使用 "Letterbox" 算法（等比例缩放并在边缘填充灰色条）以防止图像变形。
 - 通道及格式转换：
 - 将 OpenCV 默认的 BGR 格式转换为 RGB 格式。
 - 将图像维度从 `[高度, 宽度, 通道]` (HWC) 调整为 PyTorch 支持的 `[通道, 高度, 宽度]` (CHW)，并增加批次维度变成 `[批次大小, 通道, 高度, 宽度]` (NCHW)。
 - 归一化 (**Normalization**)：将图像的像素值从 $[0, 255]$ 整数缩放到 $[0.0, 1.0]$ 的浮点数。
 - 数据传输：将处理好的图像数据从 CPU 内存（Host）拷贝到 GPU 显存（Device，如果使用 CUDA 加速）。
 - Inference (推理阶段 / 模型前向传播)**：这是最核心的计算阶段。将预处理好的图像张量输入到神经网络中，通过神经网络的各层计算，生成模型对目标的原始预测数据。核心操作包括：
 - 前向传播 (**Forward Pass**)：数据流经 YOLO 的主干网络（Backbone，用于提取特征）、颈部网络（Neck，用于融合多尺度特征）和头部网络（Head，用于预测位置和类别）。
 - 矩阵乘法与卷积计算：模型在 GPU/CPU 上进行大量的卷积、激活函数（如 SiLU）、注意力机制等数学运算。
 - 输出原始预测：这一阶段结束时，模型会输出数千个候选框的坐标、置信度以及类别概率的原始张量。这是最消耗计算资源（算力）的阶段。
 - Postprocess (后处理阶段)**：将模型输出的、庞大且杂乱的原始预测张量，过滤和整理成人类和应用程序可直接使用的结构化检测结果（如干净的边界框坐标和类别）。核心操作包括：
 - 坐标解码 (**Decoding**)：将模型预测的相对比例坐标或锚框偏移量，转换回对应输入图像的绝对像素坐标（即 `x1, y1, x2, y2`）。
 - 置信度过滤 (**Confidence Thresholding**)：剔除掉所有置信度低于设定阈值（例如代码中的 `conf=0.25`）的候选框，过滤掉背景噪声。
 - 非极大值抑制 (**NMS, Non-Maximum Suppression**)：最关键的一步。由于模型可能会对同一个物体预测出多个重叠的候选框，NMS 会对比这些框的重合度（IoU），保留置信度最高的一个框，并删除与之高度重叠的其它冗余框。
 - 格式化输出：将最终留下的检测框信息从 GPU 传回 CPU，封装成易于读取的格式（如 `results.boxes`）。

1.4 Aelos 机器人 SSH 基础操作

阅读 Aelos Smart 用户使用手册，完成一次机器人基础连接操作。

要求：

- 使用 SSH 连接 Aelos 机器人；
- 通过 SSH 查看机器人中的文件目录；
- 尝试执行至少 2 个基础 Linux 命令，例如：

```
pwdls
```

4. 简要说明 SSH 连接机器人的作用：为什么机器人调试时需要远程登录？
提供以下验证材料：

- SSH 连接成功截图 1 张；
- 通过 SSH 查看机器人文件目录的截图 1 张；
- 简短文字说明：SSH 在机器人调试中的作用。

基础作业提交要求

请将基础作业整理为 一份 **PDF**，包含：

1. 环境验证和课堂 Demo 复现截图；
2. 多张图片的 YOLO 检测结果图；
3. 足球识别稳定性分析；
4. YOLO 输出字段理解题的回答；
5. Aelos 机器人 SSH 基础操作截图与说明。